# The discussion of the advantages and disadvantages of each of the 5 task scheduling algorithms:

Mounia Touil

- **First Come First Serve (FCFS)** is the simplest and most basic algorithm. It assigns tasks to VMs in round-robin without considering any task sizes or VMs speed. The advantage is that it is very easy to implement, it does not need any complex logic and tasks are executed in the order after they arrive. But it also has many disadvantages. It is very slow performance and high waiting time is does not consider task size or VM speed. This causes poor load balancing and long waiting times. Making it the least efficient algorithm of all the 5.

- **Shortest Remaining Time** improves performance better compared to FCFS. In my result, it had better throughput (0.0384) and lower turnaround time (172.35). This is because it runs small tasks first. It calculates the load on each VM based on the tasks already assigned to. The advantage of this algorithm is that it tries to balance the workload between all VMs, which prevents one VM from becoming overloaded while others are free. It also takes into account the processing speed of each VM when calculating the load, which makes it better and smarter than FCFS. But the main disadvantage is "starvation". If small tasks keep coming into the system, the big tasks might never get a chance to run and will just wait forever. So it is better than FCFS, but still not perfect.

- **Min-Min** performed better in our project. It assigns tasks to the fastest VMs that give the minimum completion time. In my result, it had a higher throughput (0.0421) and lower Turnaround Time (163.68). So the advantage that it helps reduce overall execution time and gives good throughput. It works well when there are many small tasks in the system. However, the disadvantage is that long tasks may wait a long time because short tasks always get priority first. It also requires knowing the execution time of every task on every VM before scheduling begins.

- **Max-Min** algorithm gave the best performance in this experiment. It has the highest throughput (0.0458) and the lowest Turnaround time (163.01). This is because it schedules larger tasks first while still using fast VMs, which helps balance the load better. The disadvantage is that short tasks may have to wait longer since the longest tasks always get priority. Even though it has some disadvantages, it performs best in our project.

- **Sufferage** is the smartest algorithm because it makes decisions based on how much each task would suffer if it does not get its preferred VM. The advantage is that it makes very intelligent assignment decisions by considering how much each task actually needs its best VM. The disadvantage is that it is one of the most complex algorithms to implement. With only 10 cloudlets and 3 VMs in our project, it gave a higher turnaround time (186.44), even worse than some other algorithms.

## Conclusion:

After running all the 5 algorithms, the results show a clear pattern. The two basic and simple algorithms, “FCFS” and “Shortest Remaining Time”, are easy to implement but perform poorly because they make scheduling decisions one task at a time without looking at the full picture ( task size or VM speed ). The three batch algorithms, Min-Min, Max-Min, and Suffrage, all performed significantly better because they consider all tasks and all VMs together before making any assignment.

Overall, this project shows that advanced scheduling algorithms are more effective than simple ones and should be preferred. But if simplicity and low implementation cost are the priority, and the workload is small and uniform, FCFS or Shortest Remaining Time may still be acceptable choices.